

ACTIVATION SPARSIFICATION IN TRANSFORMERS: PRESSURE, THRESHOLDING, AND SITE PLACEMENT

Anonymous authors

Paper under double-blind review

ABSTRACT

Activation sparsity can accelerate language-model inference when specialized kernels skip computation associated with zero activations. Existing methods obtain sparsity through different combinations of training-time pressure, thresholding, and placement, but their relative and combined effects have not been isolated under a common setup. As a result, it remains unclear which interventions drive the gains, when they are complementary or redundant, and how much model-wide computation they actually expose to sparsification. We study these interventions under matched pretraining conditions across Pythia models. To measure their computational effect, we define model-wide activation sparsity $\mathcal{S}_{\text{model}}$ as the fraction of forward matrix-multiplication products with a zero activation operand, together with an architecture-dependent sparsity ceiling $\mathcal{S}_{\text{model}}^{\max}$. We find that broader thresholding produces more model-wide sparsity than local FFN pressure, while combining thresholding with pressure yields the highest sparsity. The combined interventions reaches 27.48%, 40.60%, and 80.62% $\mathcal{S}_{\text{model}}$ at 14M, 70M, and 410M parameters, corresponding to 0.918, 0.822, and 0.924 of their respective sparsity ceilings. Trained recipes also dominates uniform post-hoc thresholding in the quality-sparsity frontier across all model sizes. Finally, specialized kernels achieve a $1.78\times$ full-model speedup on an RTX5090 at 14M, and speedup across 30 model checkpoints is strongly associated with $\mathcal{S}_{\text{model}}$ ($R^2 = 0.817$). These results show that activation sparsification is governed by the interaction between how zeros are induced, where they are placed, and how much of that zero structure can be exploited by sparse execution.

1 INTRODUCTION

Activation sparsity can improve language-model inference efficiency when specialized kernels skip computations associated with zero activations. Recent work has shown that sparsity can be exposed in pretrained models through post-hoc thresholding or induced during training, in both cases with measured inference speedups (Liu et al., 2024; Cetin et al., 2026). These results raise a practical design question: how should a researcher place and combine sparsification interventions to obtain more useful sparsity at an acceptable quality cost?

Existing approaches induce activation sparsity in different ways. TEAL applies post-hoc thresholding to pretrained models, ProSparse uses L1 regularization on FFN activations during training, and Q-Sparse applies top-k sparsification to both FFN and attention activations (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). While each approach is effective in its own setting, it remains unclear how they compare under a common setup and how their effects interact when combined.

In this work, we study three sparsification interventions: (i) *pressure* through ℓ_1 regularization on activations; (ii) *thresholding*, i.e. mapping small activations exactly to zero; and (iii) *placement*, which determines where these interventions are applied. To measure its effects, rather than report sparsity locally at individual activation sites, we define model-wide activation sparsity, $\mathcal{S}_{\text{model}}$, as the fraction of forward matrix-multiplication products with a zero activation operand. We additionally derive an architecture-dependent sparsity ceiling, $\mathcal{S}_{\text{model}}^{\max}$, corresponding to the maximum sparsity that can be achieved through the intervention set. Under matched pretraining conditions, we try to isolate the individual and interaction effects of each intervention and connect them to the amount of model-wide computation potentially avoided.

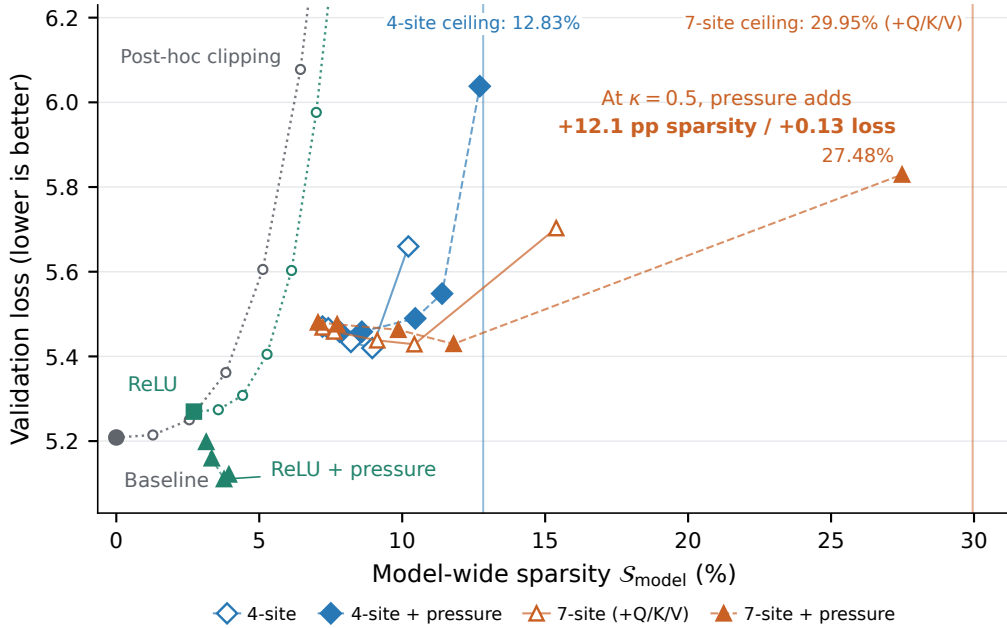


Figure 1: **Quality-sparsity trade-offs for Pythia-14M.** Markers show 26 trained checkpoints (naive L1 omitted). Four-site thresholds act at FFN and attention projection inputs; seven-site thresholds add Q/K/V. Solid and dashed curves connect trained settings without and with orthogonal L1 pressure. Gray and green dotted curves show post-hoc clipping of baseline and ReLU, respectively, at the four sites. Vertical guides show the corresponding ceilings; the 27.48% endpoint reaches 91.8% of the seven-site ceiling. Lines join evaluated settings. Six of ten evaluations per clipping path lie in the plotted range; Appendix D.3 gives the full trajectories.

We pretrain Pythia-family models from scratch for controlled comparisons across model sizes (Biderman et al., 2023). Across 30 conditions at 14M, combining thresholding with pressure raises model-wide sparsity from 15.39% to 27.48%, near its 29.95% ceiling, for +0.13 validation loss (Figure 1). FFNs are sparser at moderate thresholds, while stronger thresholding also sparsifies attention. Selected combinations outperform aggressive post-hoc clipping at 70M and 410M. ReLU pretraining also improves subsequent clipping, with larger gains at larger sizes under the same clipping setting.

Finally, we use GPT-6-Astra (OpenAI, n.d.) to develop specialized kernels and test whether $\mathcal{S}_{\text{model}}$ translates into faster inference. Across 42 iterations, the selected implementation reaches a $1.78\times$ full-model speedup over native PyTorch/SDPA on an RTX5090 for Pythia-14M at 27.48% model-wide sparsity, near its 29.95% ceiling. Across all 30 14M models, speedup is linearly correlated with $\mathcal{S}_{\text{model}}$ ($R^2 = 0.817$). Mapping architectural and train-time interventions to speedup therefore requires kernel development, which coding agents can support (Luo et al., 2026).

2 RELATED WORK

2.1 SPARSIFICATION INTERVENTIONS

Post-hoc thresholding removes small activations at inference-time of a pre-trained model. While CATS thresholds small-magnitude activations within FFNs (Lee et al., 2024), TEAL extends it to attention (Liu et al., 2024). Although effective, these studies motivate training-time interventions to adapt activation distributions to sparse computation and improve over the quality-sparsity trade-off frontier.

ReLU Strikes Back studies the effect of activations functions in language models (Mirzadeh et al., 2023), showing clear sparsity gains on the use of ReLUs. ProSparse combines progressive ℓ_1 pres-

sure and thresholding at FFNs during continued training (Song et al., 2024). It does not, however, extend its interventions to attention and restrict the analysis to local FFN sparsity. Cetin et al. (2026) extends ProSparse setup to pre-training and propose specialized kernels. These studies show that training can reshape the quality–sparsity trade-off, but they’re restricted to a single local intervention at FFN sites.

Q-Sparse extend sparsification beyond FFNs. It applies top- k selection to both attention and FFN projection inputs (Wang et al., 2024). Its top- k operation introduces, however, additional overhead due to the sorting operation. Spark Transformer tries to address it with statistical top- k (You et al., 2025), but we take a simpler approach and use a constant thresholding non-linearity. We argue that when the intervention is introduced during training, the network can adapt itself to the pressure being applied. Additionally, we evaluate the complementary effect of regularization pressure on both attention and FFNs.

2.2 INFERENCE SPEEDUP

Turning activation sparsity into speedup requires a specialized kernel adapted to the model architecture. TEAL accelerates batch-one autoregressive decoding, reporting up to $1.8\times$ speedup at 50% sparsity (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication with their TwELL kernel, and report $1.3\times$ speedup for batch inference. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You et al., 2025). In each setting, skipped computation and memory access must compesate the additional overhead introuced by the sparse operations. This development has been increasingly accelerated by the use of coding agents, through iterative kernel development with execution and profiling feedback (Ouyang et al., 2025; Luo et al., 2026). We take a related approach and use GPT-6 Astra do develop a specialized kernel based on our training interventions (OpenAI, n.d.).

3 METHODOLOGY

3.1 SPARSIFICATION INTERVENTIONS

A sparsification intervention specifies (i) where to threshold activations (thresholding sites $\mathcal{N}$), (ii) the thresholding function $G(\cdot)$, and (iii) where to apply activation pressure (pressure sites $\mathcal{P}$). The sets $\mathcal{N}$ and $\mathcal{P}$ need not coincide, and different sites may use different functions G . These choices give a combinatorial set of interventions during training.

To avoid the overhead of exact top- k selection (Wang et al., 2024; You et al., 2025), we extend ReLU to one-sided and symmetric thresholding: $G_{+,\kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm,\kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. The threshold κ controls thresholding strength and is fixed within each training condition. Appendix A gives boundary and derivative conventions.

Figure 2 locates the intervention sites on the Pythia architecture: a and m are the normalized inputs to the attention and FFN branches; h is the FFN hidden activation; z is the concatenated attention context before output projection; and q, k, v are the query, key and value operands, with q, k taken after RoPE. The attention probabilities are denoted by p . We apply $G_{+,\kappa}$ at selected sites among a, m, h, z and $G_{\pm,\kappa}$ at q, k, v . At h , the selected nonlinearity replaces GELU.

For optimization pressure, $\mathcal{L}_1$ averages mean absolute activations across targeted sites and layers, after any thresholding. *Naive LI* (LIN) minimizes $\mathcal{L}_{\text{task}} + \lambda\mathcal{L}_1$, introducing a weight λ that affects sparsity and quality (Song et al., 2024; Cetin et al., 2026). To reduce this additional tuning dimension, we propose *orthogonal LI* (OL1), inspired by gradient-conflict projection methods (Yu et al., 2020; Hsieh et al., 2024). OL1 chooses the step that maximizes alignment with the preconditioned pressure direction, within a unit norm budget relative to the adaptive Adam direction and without opposing it. Once the norm budget binds, further increasing λ leaves the ideal correction unchanged at a fixed optimizer state. To keep the multisite intervention study tractable, we use a common $\lambda = 1$ and budget $b = 1$, and directly measure whether this budget is active during training. Section 4.2 shows that budget saturation is strongly target-set dependent rather than universal. Appendix A.2 gives the formulation.

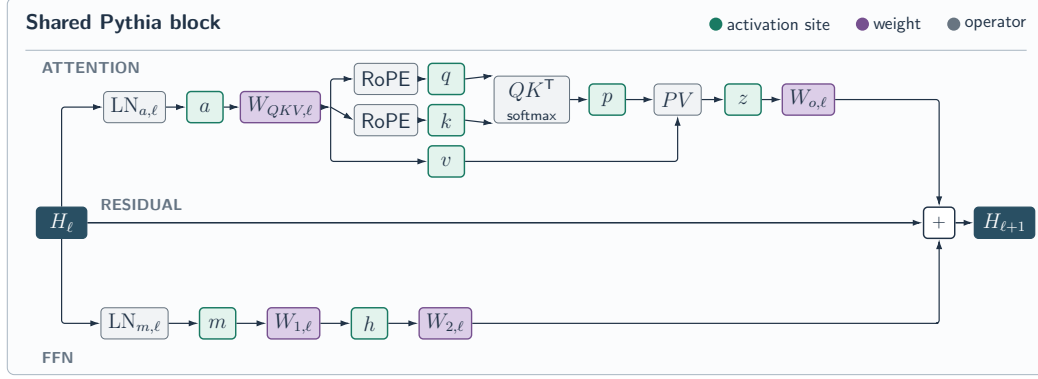


Figure 2: **Intervention sites in a shared Pythia block.** Attention and FFN branches use separate LayerNorms and join the residual path. Green nodes identify activation sites; purple nodes identify learned weights. Table 1 specifies thresholding and pressure assignments.

3.2 SPARSITY MEASURE AND ARCHITECTURAL REACH

Activation sparsity is the fraction of zero activations at a site. To report *model-wide sparsity*, we define $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, where N_0 counts scalar multiplications with a zero activation operand in the Pythia-block matrix products. N_{model} counts all scalar multiplications within those blocks and the final output projection. We pool validation counts, excluding masked attention pairs (Appendix B).

To quantify intervention potential a priori, we define the *model-wide sparsity ceiling* as $\mathcal{S}_{\text{model}}^{\text{max}} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$. It measures the fraction of scalar multiplications guaranteed a zero operand when all selected sites are 100% sparse. It depends on architecture, sites, and sequence length. Table 1 illustrates the higher ceilings of larger models at fixed sequence length. Natural zeros outside this reach can make $\mathcal{S}_{\text{model}}$ exceed $\mathcal{S}_{\text{model}}^{\text{max}}$. Appendix B.2 gives the precise accounting.

4 EXPERIMENTAL STUDY

We pretrain randomly initialized Pythia-14M, 70M, and 410M models (Biderman et al., 2023). To fit our compute budget, we adapt Pythia’s training recipe to MiniPile (Kaddour, 2023), using 1.493 billion input tokens per model. We conduct extensive comparisons at 14M and promote the sparsest recipes to 70M and 410M, alongside their baselines. We measure validation cross-entropy loss and $\mathcal{S}_{\text{model}}$ over all complete sequences from MiniPile’s validation documents. Table 1 maps the eight recipes onto the sites in Figure 2.

Table 1: **Sparsification recipes and architectural reach.** Ceilings are $100 \cdot \mathcal{S}_{\text{model}}^{\text{max}}\%$ at $T = 2,048$. A1-H recipes use ReLU at h ; A4/A7 use the thresholding functions in Section 3.1. A1-H-L1 uses L1N; the OL1 suffix denotes orthogonal pressure. All recipes are evaluated at 14M; 70M and 410M use A0, A1-H, A4-OL1 and A7-OL1.

ID	Thresholding sites $\mathcal{N}$	Pressure sites $\mathcal{P}$	Ceiling (%)		
			14M	70M	410M
A0	$\emptyset$	$\emptyset$	0.00	0.00	0.00
A1-H	h	$\emptyset$	4.28	12.35	24.93
A1-H-L1	h	h	4.28	12.35	24.93
A1-H-OL1	h	h	4.28	12.35	24.93
A4	a, m, h, z	$\emptyset$	12.83	37.06	74.78
A4-OL1	a, m, h, z	a, m, h, z	12.83	37.06	74.78
A7	a, m, h, z, q, k, v	$\emptyset$	29.95	49.42	87.25
A7-OL1	a, m, h, z, q, k, v	a, m, h, z, q, k, v	29.95	49.42	87.25

The experimental setup contains 30 training runs at 14M and 12 at each larger model size. Within each size, runs share initialization, data order and training budget. We vary local pressure for A1-H using L1N and OL1 with $\lambda \in \{0.05, 0.1, 0.5, 1\}$. For A4/A7, we sweep $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$, both without pressure and with OL1 at $\lambda = b = 1$. Figure 4 characterizes the resulting cap activity at 14M. These sweeps test which interventions yield the largest sparsification gains. Appendix C.4 details the training settings.

To compare the interventions with post-hoc clipping, we apply uniform percentile magnitude-clipping targets $p = 0, 0.1, \dots, 0.9$ at a, m, h, z as a TEAL-style baseline (Liu et al., 2024). Site-layer thresholds are calibrated separately on ten training blocks.

4.1 QUALITY-SPARSITY TRADE-OFFS

Local pressure and broader nonlinearities offer different quality-sparsity trade-offs (Figure 1). With OL1 at h , the model can improve validation loss while introducing modest model-wide sparsity: at $\lambda = 0.5$, A1-H-OL1 reaches 3.768% sparsity at loss 5.1102, compared with A0’s loss of 5.2086. A0 post-hoc clipping reaches a similar sparsity level, 3.832% at target $p = 0.3$, but with loss 5.3619. Thus, similar aggregate sparsity can carry different quality costs depending on the trained recipe and evaluation-time intervention.

Broader interventions reach much greater sparsity. A7-OL1 at $\kappa = 0.5$ attains 27.483% model-wide sparsity at loss 5.8294, accepting a quality cost relative to A0. This comparison also changes computational reach: the A0 clipping reference acts at a, m, h, z , whereas A7 additionally thresholds the internal attention operands. At 14M, these site sets have analytic sparsity ceilings of 12.833% and 29.952%, respectively. This comparison changes both the trained representation and the computation the selected sites can affect. Matched pressure-on/off comparisons isolate pressure’s contribution. Full endpoints and clipping trajectories are given in Appendix D.

In the local h -only sweep, OL1 is not uniformly better than naive L1: their validation-loss differences are small, and the ordering reverses at $\lambda = 1$; the complete matched sweep is reported in Appendix D, Table 6.

4.2 PAIRED EFFECT OF ADDING PRESSURE

To isolate the marginal contribution of pressure from threshold strength, we compare each OL1 recipe with its pressure-free counterpart at the *same* trained threshold κ (Figure 3). Thus, the four-site contrasts are A4-OL1 minus A4 and the seven-site contrasts are A7-OL1 minus A7; changing κ changes the operating point at which pressure is evaluated, while retaining a pressure-free reference at each threshold.

The added value of pressure changes sharply with both threshold and target set. For the four-site recipe, OL1 slightly improves validation loss at $\kappa = 0$ and 0.01, while adding 0.598 and 1.173 percentage points (pp) of model-wide sparsity. At larger thresholds, the sparsity increment approaches 2.5 pp but carries increasing quality cost, reaching +2.498 pp for +0.3783 validation loss at $\kappa = 0.5$. For the seven-site recipe, the pressure effect remains modest through $\kappa = 0.1$, where it adds 1.372 pp for only +0.0008 loss. At $\kappa = 0.5$, however, the increment rises to +12.096 pp for +0.1265 loss.

At the highest threshold, seven-site pressure therefore produces nearly five times the additional model-wide sparsity of four-site pressure, while incurring roughly one third of its validation-loss increase.

This difference characterizes the complete pressure target sets, not an isolated effect of adding q, k, v . Seven-site OL1 introduces additional pressure targets, and because the pressure objective equally averages targeted site-layer tensors, expanding the target set also changes the normalization and relative weighting of the existing targets. The larger seven-site effect therefore cannot be attributed to attention placement alone.

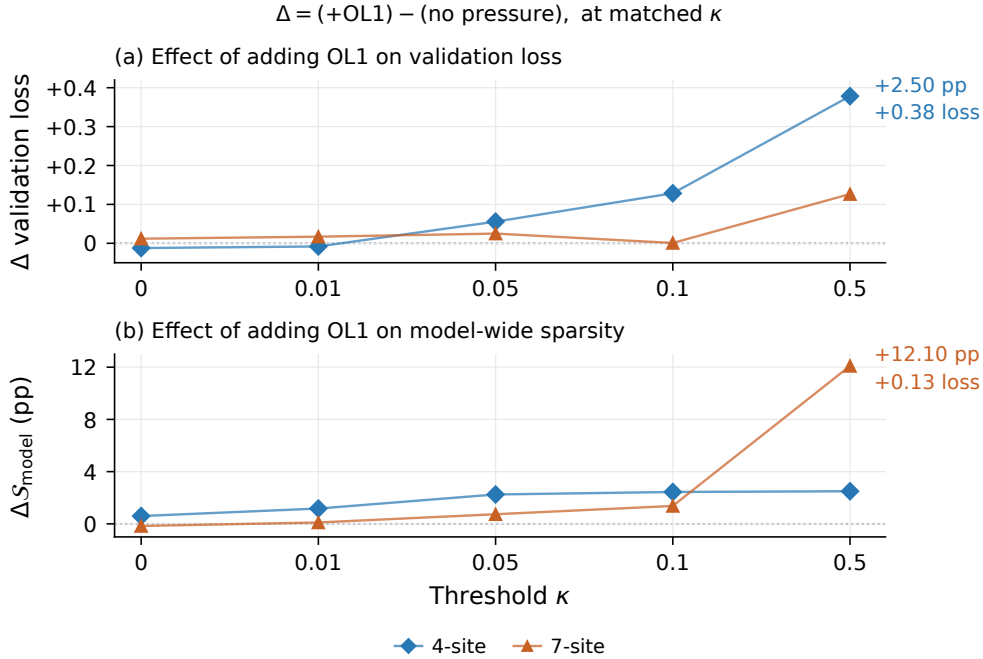


Figure 3: **The marginal effect of adding OL1 depends strongly on threshold and pressure target set.** Each point is a matched comparison at the same trained threshold κ : $\Delta = (+\text{OL1}) - (\text{no pressure})$, using A4-OL1 – A4 for the four-site recipe and A7-OL1 – A7 for the seven-site recipe. (a) Change in validation loss; negative values indicate improved quality. (b) Change in model-wide sparsity S_{model} , in percentage points. **At $\kappa = 0.5$, four-site pressure adds 2.50 pp sparsity for +0.38 loss, whereas seven-site pressure adds 12.10 pp for only +0.13 loss.** Lines connect matched threshold settings and do not represent training trajectories. All comparisons use matched initialization, data order, and training budget.

The same pressure weight produces very different optimization regimes across target sets (Figure 4). We log global dot products and norms of the AdamW-preconditioned task direction u and pressure direction w at every optimizer step of the ten Pythia-14M multisite OL1 runs. Pre-projection alignment is negative on 99.9% of the resulting 7,120 step records. We quantify the task-relative opposing component as

$$\rho_{\text{opp}} = \frac{\max(0, -\langle u, w \rangle)}{\|u\|_2^2 + \epsilon},$$

which expresses the removed opposing component relative to the task-direction norm. Pooled across thresholds, its median is 0.0095 for four-site pressure and 0.588 for seven-site pressure. Consistently, at $\lambda = b = 1$, the norm cap is active on only 0.5% of four-site steps but 97.0% of seven-site steps. Thus, seven-site OL1 operates almost entirely in a budget-limited regime, whereas four-site OL1 is almost entirely uncapped. Because expanding to seven sites also changes the pressure objective and its equal-tensor normalization, these diagnostics characterize the complete target sets rather than isolating the contribution of q, k, v alone.

Pressure therefore cannot be assigned a single context-independent sparsity–quality effect: its marginal contribution depends on the thresholded representation and pressure target set.

4.3 BROADER THRESHOLD PLACEMENT WITHOUT PRESSURE

To isolate the effect of adding Q/K/V thresholding, we compare A7 with A4 at matched κ without pressure (Table 2). **At $\kappa = 0.5$, adding Q/K/V thresholding raises model-wide sparsity by 5.17 pp for only +0.043 validation loss.**

We next examine how the resulting four- and seven-site recipes differ in their activation structure.

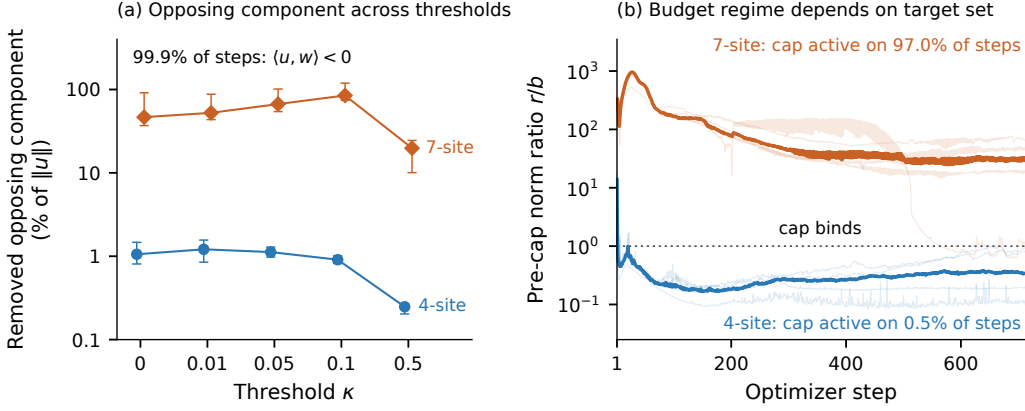


Figure 4: **OL1 pressure geometry depends strongly on the pressure target set.** (a) Task-relative magnitude of the opposing pressure component removed by conflict-conditioned projection. For each trained threshold κ , points show the median over 712 optimizer steps of $100\rho_{\text{opp}}$, where $\rho_{\text{opp}} = \max(0, -\langle u, w \rangle) / (\|u\|_2^2 + \epsilon)$; whiskers show the 25th–75th percentiles. Across all 7,120 logged multisite OL1 steps, 99.9% have $\langle u, w \rangle < 0$. (b) Pre-cap pressure-to-task norm ratio r/b at $\lambda = b = 1$. Faint curves show the five κ conditions within each target set, and thick curves show their per-step median. The dotted line marks the nominal cap boundary $r/b = 1$; cap-active fractions use the implemented condition $s < 1$. The cap is active on 0.5% of four-site steps and 97.0% of seven-site steps. All quantities are measured in the AdamW-preconditioned coordinates used by OL1, before group learning rates and decoupled weight decay. Thus, expanding the pressure target set changes not only where pressure is applied but also the task-relative magnitude and budget regime of the pressure correction.

Table 2: **Broader threshold placement increases model-wide sparsity at matched threshold without pressure.** A4 thresholds a, m, h, z , whereas A7 additionally thresholds q, k, v . At $\kappa = 0$, the additional Q/K/V nonlinearities are identities, and the two recipes are nearly identical. As κ increases, their sparsity difference grows; at $\kappa = 0.5$, A7 reaches 15.39% model-wide sparsity versus 10.22% for A4, with validation losses 5.7029 and 5.6597, respectively.

κ	A4 loss	A7 loss	A4 S_{model} (%)	A7 S_{model} (%)
0	5.4705	5.4684	7.212	7.218
0.01	5.4665	5.4588	7.414	7.618
0.05	5.4341	5.4379	8.206	9.127
0.1	5.4197	5.4287	8.953	10.425
0.5	5.6597	5.7029	10.216	15.387

4.4 LOCAL SPARSITY DOES NOT DETERMINE MODEL-WIDE SPARSITY

FFN sparsity alone does not determine which recipe exposes more model-wide sparse computation (Figure 5). At $\kappa = 0.5$, four-site OL1 has 99.31% exact zeros in the pooled FFN activations, compared with 93.64% for seven-site OL1. However, seven-site OL1 has the larger model-wide sparsity: 27.48% versus 12.71%. It also has lower validation loss at this operating point, 5.8294 versus 6.0380. **Thus, the recipe with the larger FFN zero fraction is not the recipe with the larger $\mathcal{S}_{\text{model}}$.**

The operation decomposition explains the difference. Under four-site OL1, QK and PV together contribute only 0.06 percentage points to $\mathcal{S}_{\text{model}}$; under seven-site OL1 they contribute 16.77 points. This increase more than offsets the smaller seven-site contributions from QKV and FFN-up projections. **Model-wide sparsity therefore depends on both the zero fraction at each activation site and the amount of matrix-product work that consumes those zeros.**

The nonzero density also shows why concentration near zero should not be interpreted as exact sparsity. Under four-site OL1, the pooled Q/K/V distribution is sharply concentrated near zero, but only 0.22% of those activations are exactly zero. Under seven-site OL1, 95.60% are exactly zero. **A narrow near-zero distribution therefore need not correspond to a large amount of zero-skippable computation.**

Q/K/V receive neither direct thresholding nor direct pressure in the four-site recipe, yet their distributions differ substantially from the untreated reference (Appendix D.1, Figure 9). This shows that changing the four-site recipe is associated with changes at activation sites that are not directly targeted. We refer to this as a **nonlocal response of the trained network**. The comparison does not isolate whether that response is caused by pressure, thresholding, or their interaction.

Expanding the intervention to Q/K/V also changes the FFN distribution: pooled FFN exact-zero mass decreases from 99.31% under four-site OL1 to 93.64% under seven-site OL1. This shows that adding intervention sites does not simply add sparsity independently at those sites; the activation distributions elsewhere in the network also change. Because the seven-site recipe changes both the pressure target set and the normalization of the pressure objective, this comparison does not identify which component causes the FFN change.

4.5 TRANSFER ACROSS MODEL SIZES

At $\kappa = 0.5$, seven-site + OL1 has both lower validation loss and greater model-wide sparsity than four-site + OL1 at all three model sizes (Figure 6). The same ordering does not hold at $\kappa = 0$: four-site + OL1 is better on both axes at 14M and 70M, whereas seven-site + OL1 is better on both axes at 410M. Thus, the cross-size result is recurrence of the **high-threshold complete-recipe relationship**, not transfer of the entire threshold response. The larger cohorts lack the pressure-free A4/A7 controls needed to isolate pressure’s contribution. Exact matched seven-site minus four-site OL1 contrasts at all five thresholds are reported in Appendix Table 7.

Raw $\mathcal{S}_{\text{model}}$ must be interpreted relative to the computation reachable by each architecture. The seven-site sparsity ceiling increases from 29.95% at 14M to 49.42% at 70M and 87.25% at 410M, largely because the dense output projection accounts for a smaller fraction of the counted products as model width and depth increase. At $\kappa = 0.5$, seven-site + OL1 reaches 27.48%, 40.60%, and 80.62% $\mathcal{S}_{\text{model}}$, corresponding to **91.8%, 82.2%, and 92.4% of the respective seven-site ceilings**. The much larger raw sparsity at 410M therefore reflects in part a larger architectural opportunity for the selected intervention sites, rather than an isolated effect of model size on learned zero fractions.

Absolute validation loss is not monotonic with model size under the realized protocol: the A0 end-points are 5.2086, 4.0998, and 4.5475 for 14M, 70M, and 410M. This experiment holds the input-token budget fixed rather than tokens per parameter; the resulting budgets are 106.14, 21.20, and 3.68 tokens per parameter, and 410M also uses a lower peak learning rate. Appendix Figure 8 reports the corresponding A0 optimization trajectories. **We therefore use the size sweep to test whether within-size recipe relationships recur, not as a language-model quality scaling law.**

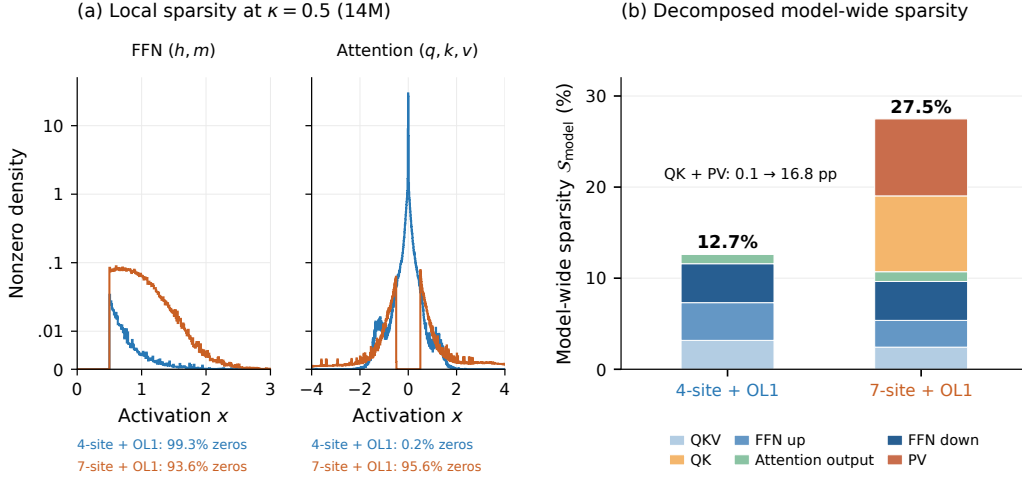


Figure 5: **Local activation sparsity does not determine model-wide sparsity.** (a) Nonzero activation densities for four-site and seven-site OL1 at $\kappa = 0.5$ in Pythia-14M; exact-zero mass is reported below each panel. Four-site OL1 has a larger pooled FFN zero fraction than seven-site OL1 (99.31% versus 93.64%), whereas seven-site OL1 has a much larger pooled Q/K/V zero fraction (95.60% versus 0.22%). (b) Contribution of each matrix-product family to model-wide sparsity S_{model} , using the common full-model denominator. Despite its larger FFN zero fraction, four-site OL1 reaches only $S_{\text{model}} = 12.71\%$, compared with 27.48% for seven-site OL1. QK and PV contribute 16.77 percentage points under seven-site OL1 but only 0.06 under four-site OL1. Bars report zero-operand multiplication opportunities, not runtime savings. Density pooling and normalization are detailed in Appendix D.1.

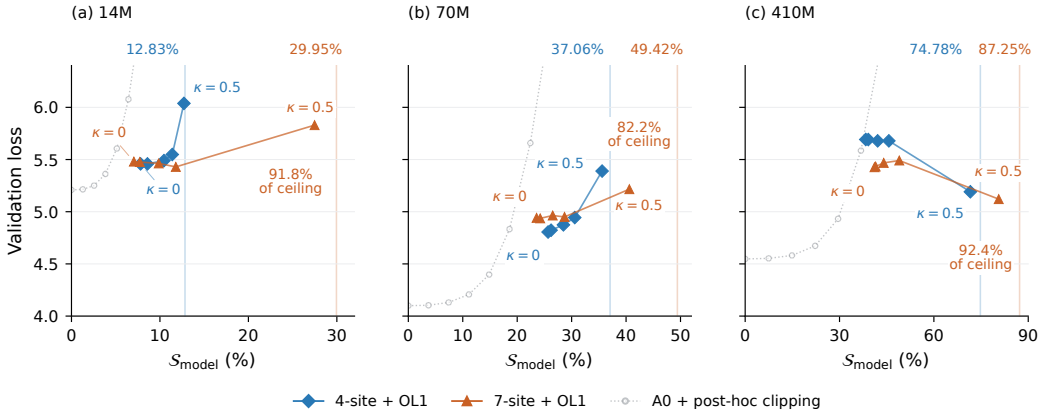


Figure 6: **The high-threshold seven-site recipe remains better on both quality and model-wide sparsity across model sizes.** Absolute validation loss versus model-wide sparsity S_{model} for Pythia-14M, 70M, and 410M. Blue and orange curves connect separately trained four-site + OL1 and seven-site + OL1 checkpoints at $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$; the gray dotted curves apply post-hoc clipping at a, m, h, z to the corresponding A0 checkpoints. Vertical guides mark the analytic four-site and seven-site sparsity ceilings, and annotations at the seven-site $\kappa = 0.5$ endpoints report the fraction of the seven-site ceiling reached. **At $\kappa = 0.5$, seven-site + OL1 has both lower validation loss and greater S_{model} than four-site + OL1 at all three sizes, reaching 91.8%, 82.2%, and 92.4% of the respective seven-site ceilings.** The ordering at $\kappa = 0$ changes with size. Lines connect evaluated settings rather than interpolated models; full post-hoc clipping trajectories, including higher-loss evaluations outside the displayed range, are reported in Appendix D.3.

4.6 REALIZING SPARSITY WITH SPECIALIZED INFERENCE KERNELS

Realizing activation sparsity requires kernels matched to the model’s shapes, nonlinearities and hardware. The released TwELL kernels (Cetin et al., 2026) accelerated their supported workload, but the unchanged small-width primitive failed numerical qualification for Pythia-14M. A shape- and sign-adapted descendant, P0, qualifies on only four of our 30 checkpoints, with geometric mean speedup $0.8910\times$ on that subset (Appendix D.4). This motivates specialization for the architecture and intervention.

To make this specialization practical within an architecture study, we use a human-guided coding-agent loop to propose, test and refine implementations. Execution and profiling feedback provide a basis for this approach (Ouyang et al., 2025; Luo et al., 2026). We replay 42 eligible proposals from the Codex search against native PyTorch scaled dot-product attention (SDPA) in CUDA graphs on one RTX5090. The workload is BF16, batch-one, uncached 2,048-token inference with full vocabulary logits. Qualification requires logits and loss to agree with the native reference within specified tolerances on complete validation.

The matched retrospective reaches $1.6024\times$ by iteration 11 and $1.7830\times$ at iteration 42 on the fixed A7-OL1, $\kappa = 0.5$ search checkpoint (Figure 7, left): substantial gains appear early, followed by a later improvement. The selected implementation K050 qualifies on all 30 checkpoints in the 14M cohort, with geometric mean speedup $1.2340\times$. Speedup has a strong, approximately linear association with model-wide sparsity (right; OLS $R^2 = 0.8167$). This connects the logical metric to measured execution, while checkpoint quality, topology and weights vary across the fit.

Ablations distinguish fusion from the contribution of sparse paths. Relative to the same fusion with sparse paths disabled, K050 is $1.311\times$ as fast at the search checkpoint and $1.043\times$ as fast geometrically across the cohort, helping 14 of 30 checkpoints. Attention skipping is unprofitable in this workload: disabling it improves every checkpoint, raising the mean to $1.2506\times$. The attention kernel can skip a matrix-multiply instruction when either complete operand fragment is zero; matching zeros in both Q and K are unnecessary. Yet even substantial instruction skipping fails to repay its overhead at 14M (Appendix D.4). The full-model gain thus combines fusion with a conditional benefit from sparse paths: profitable execution depends on both sparsity and the granularity at which work is skipped.

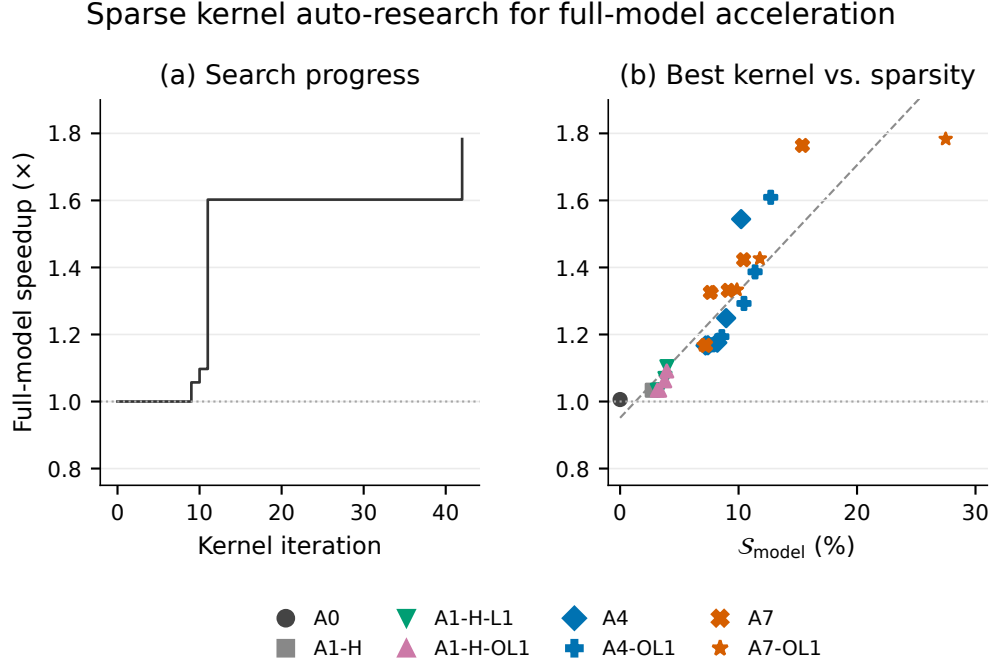


Figure 7: Sparse kernel auto-research for full-model acceleration. (a) Matched retrospective of the best qualified incumbent on fixed A7-OL1, $\kappa = 0.5$, across 42 iterations, starting at native $1\times$ and reaching $1.7830\times$. (b) The selected K050 implementation on 30 Pythia-14M checkpoints; symbols identify recipes. The dashed line is a descriptive OLS fit with intercept ($R^2 = 0.8167$); dotted lines mark native $1\times$. Speedup uses each checkpoint’s native SDPA CUDA-graph baseline on one RTX5090: BF16, batch one, uncached 2,048-token inference and full vocabulary logits. Qualification covers all 338 validation blocks. All checkpoints qualify, with geometric mean $1.2340\times$. Here “best” refers to the searched proposals; the attention-dense ablation is faster on every checkpoint (Table 10). Sparsity uses FP16 measurements, while timing uses BF16; both speedup axes have restricted ranges. The fit describes association across different checkpoints, not a causal sparsity effect.

5 DISCUSSION AND CONCLUSION

Pressure, thresholds and site placement need to be evaluated together. Site placement also changes the optimization regime: at the common $\lambda = 1$ in the 14M runs, seven-site pressure is almost always norm-limited, whereas four-site pressure is almost never capped. In the matched 14M comparisons, adding pressure ranges from improving both loss and sparsity to exchanging quality for additional zeros. Extending the intervention to internal attention can increase model-wide sparsity even when FFN sparsity falls. The high-threshold seven-site recipe with pressure improves both axes over the four-site recipe at all three sizes, but the zero-threshold ordering changes. These results favor evaluating each addition against its matched recipe, rather than choosing interventions from activation sparsity alone.

The interpretation is supported by a connected pattern across experiments. Matched comparisons identify how pressure’s added effect changes with the threshold; distribution measurements reveal which activations change; and product counts explain their model-wide consequence. The high-threshold A7-OL1 advantage recurs at all three sizes, while the zero-threshold reversals identify where that relationship stops holding. Agreement across these comparisons makes the result more informative than a favorable checkpoint or an isolated sparsity percentage.

Product accounting and kernel timing answer different questions. Counting zero-operand products shows where an intervention affects computation; the sparsity ceiling explains how architecture changes that opportunity. The kernel study confirms that some of it can reduce full-model latency, but also shows that skipping attention work can cost more than executing it. Fusion and sparse-path ablations are therefore necessary to attribute a measured speedup.

Cross-size comparisons use a common token budget rather than equal tokens per parameter, and 410M also uses a lower peak learning rate; the A0 optimization trajectories in Appendix Figure 8 show that the three sizes finish in different realized optimization regimes. Accordingly, the scale experiment tests recurrence of within-size recipe relationships rather than quality scaling with parameter count. These conclusions concern MiniPile pretraining and complete recipes at the larger sizes. The distribution diagnostics cover 14M and describe responses rather than isolating a causal spillover mechanism. The kernel results use one RTX5090, full-sequence inference, native SDPA and matched fusion controls. Transfer to cached decoding, larger models and other hardware remains untested.

REFERENCES

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.
- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.

- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- OpenAI. GPT-6 Astra. OpenAI API documentation, n.d. URL <https://developers.openai.com/api/docs/models/gpt-6-astra>. Accessed September 10, 2026.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A INTERVENTION DEFINITIONS

A.1 THRESHOLDING NONLINEARITIES

For finite $\kappa \geq 0$, both thresholding nonlinearities retain equality and leave retained values unchanged. The comparison mask is detached: the derivative with respect to the input is one for retained entries and zero for rejected entries. Consequently, $G_{+,0}$ and ReLU have identical forward values but differ at zero, where their derivatives are one and zero, respectively. $G_{\pm,0}$ is exactly the identity in both values and gradients. These nonlinearities use neither a straight-through estimator nor a top- k selection rule. Unselected sites retain their original transformations.

A.2 ACTIVATION PRESSURE AND OL1

For the J targeted site-layer tensors A_j , containing n_j entries each, the pressure objective is $\mathcal{L}_1 = J^{-1} \sum_{j=1}^J n_j^{-1} \sum_{i=1}^{n_j} |A_{j,i}|$. The tensors contain the executed site activations, after the non-linearity where one is applied. Absolute values and means are evaluated in float32. Each tensor receives equal weight regardless of its size; adding pressure sites changes both the objective and the weight of existing targets. Task and pressure losses are averaged over the same accumulation microbatches. For naive L1, the accumulated gradient of $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$ is globally norm-clipped before AdamW (Loshchilov & Hutter, 2019).

For OL1, let g_t and g_1 be the accumulated task and unweighted pressure gradients. We first clip g_t by its global norm and perform AdamW using only that clipped task gradient. Let $\hat{m}_t$ and $\hat{v}_t$ be its newly updated, bias-corrected moments. On parameters having both gradients and initialized optimizer state, define the adaptive task and pressure vectors $u = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon_A)$ and $w = g_1 / (\sqrt{\hat{v}_t} + \epsilon_A)$. The pressure gradient is unclipped and has no first-moment buffer. Inner products and norms below pool all these eligible parameters. With $c = \langle u, w \rangle$, $q = \|u\|_2^2$, and numerical stabilizer $\epsilon > 0$, the conflict-conditioned projection is

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (1)$$

For diagnostics, we define the task-relative opposing component

$$\rho_{\text{opp}} = \frac{\max(0, -c)}{q + \epsilon}.$$

When projection is active, this equals the norm of the component removed from w , divided by $\|u\|_2$; it is zero when $c \geq 0$. All reported steps satisfy the guard $q > \epsilon$. It is therefore not the fraction of the pressure norm removed.

For pressure weight $\lambda \geq 0$ and required budget $b > 0$, define the norm ratio $r = \lambda \|\tilde{w}\|_2 / (\|u\|_2 + \epsilon)$. Set $s = \min\{1, b/(r + \epsilon)\}$ when $r > 0$, or $s = 1$ otherwise. After AdamW, each eligible parameter group p , with learning rate α_p , receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$. The experiments' clipping thresholds, b , and λ are condition parameters. We report the norm budget as active when the implemented scale satisfies $s < 1$.

Ignoring numerical stabilizers, the correction norm before the learning rate is $\min\{\lambda \|\tilde{w}\|_2, b\|u\|_2\}$. For a fixed optimizer state, once the cap binds, further increases in λ leave this ideal correction unchanged.

This procedure is related to gradient-conflict projection (Yu et al., 2020) and the use of smoothed primary directions (Hsieh et al., 2024), but defines a particular correction in AdamW-preconditioned coordinates. The stabilized projection is only approximately orthogonal under conflict. Its geometry and budget precede multiplication by group learning rates and exclude decoupled weight decay. They therefore do not guarantee task-loss descent or preservation.

B SPARSITY ACCOUNTING

B.1 EXACT COUNTERS

Activation sparsity at site s is $\mathcal{Z}_s = \sum_{\ell,e,i} \mathbf{1}\{A_{s,i}^{(\ell,e)} = 0\} / \sum_{\ell,e} |A_s^{(\ell,e)}|$, where ℓ indexes layers, e evaluation batches, i tensor entries, and $|A|$ the number of elements in tensor A . Near-zero mass replaces equality by $|A_{s,i}| \leq \varepsilon$ for a stated tolerance; it does not contribute to exact-zero counts unless the value is zero. All aggregate fractions pool integer numerators and denominators across batches and layers before division.

The atomic unit of model-wide accounting is one scalar multiplication in a matrix product. If $X \in \mathbb{R}^{n \times p}$ feeds a linear projection with q outputs, the count is $N = npq$, with $N_0 = q \sum_{i,j} \mathbf{1}\{X_{ij} = 0\}$. Zero weights receive no credit. For the multihead attention graph specified in Appendix C.1, both operands are activations, so a product is credited once if either is zero. Within one block and evaluation batch, for query position t , key position $u \leq t$, and within-head coordinate j ,

$$N_{0,QK} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{Q_{bhtj} = 0 \vee K_{bhuj} = 0\}, \quad (2)$$

$$N_{0,PV} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{P_{bhtu} = 0 \vee V_{bhuj} = 0\}. \quad (3)$$

Here P is the causal softmax probability tensor; the model numerator sums these counts over blocks and evaluation batches. Future masked positions are excluded from both counts, while zeros on valid positions, including floating-point underflow, are included. Counts use the operands actually consumed by each matrix product; upstream masks do not determine those operands' zeros after intervening transformations. Earlier value positions participate in more causal pairs, so local value sparsity alone does not determine PV sparsity.

B.2 SPARSITY CEILING AND NORMALIZATION

For a specified graph and workload, let N_o count the valid scalar multiplications in block operation family o , pooled over layers and examples. Then $N_{\text{model}} = \sum_o N_o + N_{\text{head}}$, while N_0 sums the corresponding zero-operand counts only in block operations. The block-only view is $\mathcal{S}_{\text{block}} = N_0 / \sum_o N_o$. Both sparsities are fractions; percentages multiply them by 100. Embedding lookup, normalization, nonlinearities, softmax, RoPE, bias and residual additions, and data movement are outside this accounting.

To construct the reach numerator, count products guaranteed to have a zero activation operand as a structural consequence of 100% activation sparsity at the selected sites, independently of checkpoint values or natural zeros elsewhere. Include exact zero-preserving propagation under the declared graph rules. Products reached through more than one site receive one credit. The resulting count is $N_{\text{reach}}(\mathcal{N})$. Division by the same N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$. Natural zeros outside this selected-site reach can contribute to observed $\mathcal{S}_{\text{model}}$, so the ceiling is not an upper bound on every observation. Its gap to an observation does not measure sparsity available without quality loss. Appendix C.2 specializes the denominator and reach rules to the Pythia graph used in the experiments.

C EXPERIMENTAL SETUP DETAILS

C.1 PYTHIA SITES AND NONLINEARITIES

Pythia uses parallel attention and FFN branches with separate LayerNorms. For batch size B , sequence length T , residual width d , FFN width d_f , and H heads, let $d_h = d/H$. Table 3 identifies the actual operands recorded after the selected nonlinearity. The same site mapping is applied in every layer, with one common threshold per condition. Figure 2 shows the shared block; Table 1 gives its thresholding and pressure assignments.

At h , the selected nonlinearity replaces GELU. Other sites retain their original activations unless a thresholding nonlinearity is applied. At zero threshold, A7's additional symmetric query/key/value nonlinearities are exact identities in both values and gradients. They therefore do not change A4's

Table 3: Intervention sites and the matrix products that consume them directly.

Site	Shape	Consuming operation	Placement
a	$B \times T \times d$	QKV projection	After attention LayerNorm
m	$B \times T \times d$	FFN up projection	After FFN LayerNorm
h	$B \times T \times d_f$	FFN down projection	Nonlinearity output
q, k	$B \times H \times T \times d_h$	Attention scores	After partial RoPE
v	$B \times H \times T \times d_h$	Probability-value	After the QKV split
z	$B \times T \times d$	Attention output	Concatenated context, before output projection

mathematical forward map in the absence of pressure. A7-OL1 still differs from A4-OL1 in its pressure targets and their equal-tensor normalization.

Query and key counts use actual post-RoPE operands: rotation can repopulate an individual zero coordinate. Likewise, the output projection uses actual post-nonlinearity context z , rather than inferring its zeros from the value mask.

C.2 ARCHITECTURE-SPECIFIC COUNTS AND SPARSITY CEILINGS

The operation inventory comprises QKV projection, attention scores (QK), probability-value products (PV), attention-output projection, and the two FFN projections. For one uncached, unpadded sequence, each block contributes $3Td^2$ products from QKV projection, Td^2 from the attention-output projection, Tdd_f from each FFN projection, and $dT(T+1)/2$ from each of QK and PV . Thus, for L blocks and vocabulary size V_{vocab} ,

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (4)$$

Over B sequences, $N_{\text{model}} = BC_{\text{model}}$; the dense head contributes only to the denominator, as defined in Section 3.2.

Let $\mathcal{O}(\mathcal{N})$ be the union of operations reachable when the selected sites have 100% activation sparsity: a reaches QKV; m reaches FFN up; h reaches FFN down; either q or k reaches QK ; v reaches PV and attention output; and z reaches attention output. Each operation is counted once. The v rule includes the exact closure $V = 0 \Rightarrow PV = 0$. No further propagation is assumed: QKV biases can survive $a = 0$, and zero scores do not imply zero softmax probabilities. For the same site set in every layer, $N_{\text{reach}} = BL \sum_{o \in \mathcal{O}(\mathcal{N})} C_o$, where C_o is that operation’s per-sequence count. Division by N_{model} gives $\mathcal{S}_{\text{model}}^{\text{max}}$.

With Pythia’s $d_f = 4d$, the per-block reachable counts for A0, A1-H, A4, and A7 are 0, $4Td^2$, $12Td^2$, and C_{block} , respectively. The ladder evaluates these formulas at $T = 2,048$, $V_{\text{vocab}} = 50,304$, and the pinned (L, d) configurations (6, 128), (6, 512), and (24, 1,024).¹ No checkpoint enters this calculation. For example, A0 has zero selected-site reach but may have positive observed sparsity from natural zeros.

C.3 EVALUATION SCOPE

Quality and logical counts are paired in the same eager-attention evaluation of a checkpoint, with its declared nonlinearities and evaluation precision. The complete MiniPile validation split contains 500 documents. Concatenation and packing produce 338 complete blocks of 2,048 tokens (692,224 input tokens); the final 1,444-token tail is excluded. Each block contributes 2,047 next-token targets to cross-entropy, while product accounting includes all 2,048 input positions. Loss is averaged over the equally sized blocks, and sparsity is count-pooled. This workload uses no key-value cache. Condition-specific training budgets, optimizer settings, seeds, and checkpoint selection accompany their results; they are not implied by the ladder’s list of candidate conditions.

¹Architecture configurations: EleutherAI’s Pythia-14M-deduped, Pythia-70M-deduped, and Pythia-410M-deduped. Links identify immutable configuration revisions; released weights are not used.

C.4 REALIZED TRAINING PROTOCOL

We use MiniPile and the Pythia-14M-deduped tokenizer for every size. Each document is encoded without added special tokens, followed by one end-of-sequence token. Concatenated tokens are packed into nonoverlapping 2,048-token blocks. Training uses a seeded permutation of the block indices; validation retains source order. The training split contains 1,000,000 documents and 1,491,711,416 tokens, yielding 728,374 complete blocks and an excluded 1,464-token tail. The supplementary `protocol.json` records immutable dataset and tokenizer revisions, cache hashes and implementation identities.

All conditions start from random weights drawn from the pinned Pythia architecture configurations; released pretrained weights are not used. Model initialization and data-order seeds are both 1234, with one training seed per condition. Parameter sweeps measure intervention variability rather than variation over independent training seeds. All quality comparisons reuse the same validation split, without a separate confirmation set. Within a size, the conditions share the initial parameter hash and training schedule. The 30-condition 14M cohort contains A0, A1-H, four weights each for A1-H-L1 and A1-H-OL1, and five thresholds each for A4, A4-OL1, A7 and A7-OL1. Each larger cohort contains A0, A1-H, and the five-threshold A4-OL1/A7-OL1 sweeps. Local pressure weights are $\{0.05, 0.1, 0.5, 1\}$; thresholds are $\{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, with pressure at all four or seven selected sites. Every OL1 condition uses trust budget 1 and projection stabilizer $\epsilon = 10^{-12}$.

Training uses AdamW with $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 and global task-gradient clipping at norm 1. Naive L1 clips the combined objective gradient instead. The learning rate warms up linearly for eight updates and follows cosine decay to one tenth of its peak. Parameters and optimizer state are FP32; forward computation uses FP16 autocast with dynamic loss scaling. Attention and hidden dropout are zero. Table 4 gives size-dependent settings. The global batch is 1,024 sequences of 2,048 tokens, for 712 optimizer updates and 1,493,172,224 input tokens. This wraps 714 blocks beyond the 728,374 complete blocks in one packed training pass. We use the final checkpoint without validation-based checkpoint selection. The lower 410M learning rate is part of the realized protocol, so size alone is not the only cross-scale change.

Table 4: Size-dependent training settings. Tokens per parameter use the common realized input-token budget; they are not held fixed across sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation	32	256	256
Tokens per parameter	106.14	21.20	3.68

Figure 8 shows the A0 optimization trajectories under the common token budget. Training loss trends downward across the runs, but the three sizes finish in different optimization regimes: 70M reaches the lowest training loss under the realized schedule, while 410M finishes above it and exhibits a larger pre-clipping task-gradient norm late in training. These observations are **consistent with different degrees of optimization under the fixed-token protocol**, but they do not establish whether any run has or has not converged. The trajectories therefore provide context for the non-monotonic A0 validation losses in Figure 6 rather than a basis for extrapolating performance with additional training.

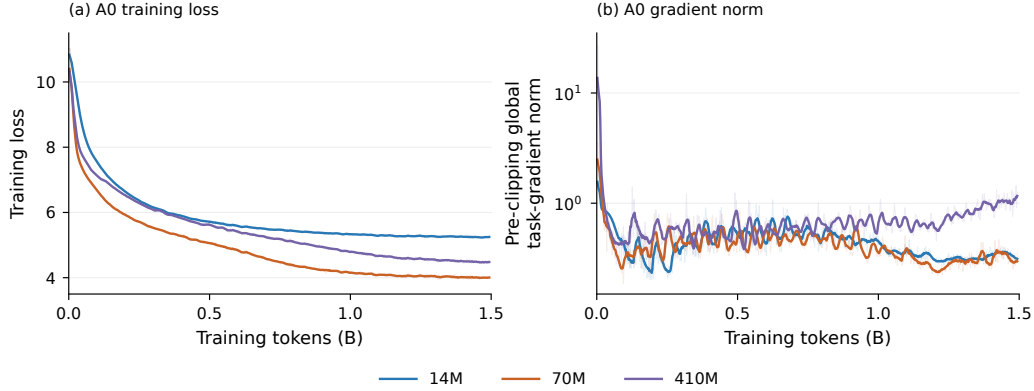


Figure 8: **A0 optimization trajectories under the common training-token budget.** (a) Training loss as a function of consumed input tokens for the 14M, 70M, and 410M A0 runs. (b) Pre-clipping global task-gradient norm over the same token budget; the logarithmic y-axis shows the large initial gradient transient and subsequent training regime. Faint traces show all 712 logged optimizer steps per size; opaque curves show a centered nine-step arithmetic moving mean, with partial windows at the edges. Norms are measured after loss unscaling and gradient accumulation, before clipping; smoothing is applied in the original units. All three sizes receive approximately 1.493B input tokens, but this corresponds to progressively fewer tokens per parameter as model size increases; the 410M run also uses a lower peak learning rate. Global norms are not normalized by parameter count. These trajectories characterize the realized optimization regimes and are not used as a convergence criterion.

D COMPLETE RESULTS AND SUPPORTING DIAGNOSTICS

The following tables report every selected trained endpoint, all 29 paired 14M contrasts and all 15 cross-scale recipe comparisons. They retain dominated settings as well as favorable ones. Each entry summarizes one trained model on complete validation. Sparsity percentages are formed from pooled integer product counts. The endpoint table uses the common A7 sparsity-ceiling denominator from Figure 6, rather than a different denominator for each recipe. Here $U_{\text{arch}} = \mathcal{S}_{\text{model}} / \mathcal{S}_{\text{model}}^{\text{max}}(\text{A7})$ denotes this common-reference ratio, retained for the complete endpoint table.

D.1 ACTIVATION SPARSITY AND SIGNED-DENSITY COVERAGE

The seven frozen-checkpoint evaluations in Figure 9 record signed values after the trained nonlinearities, with Q/K measured after RoPE. Each checkpoint contributes 2,658,140,160 FFN elements and 1,594,884,096 attention elements across the complete validation set. Summing integer counts gives the 80:20 weighting of $h : m$ and equal weighting of $q : k : v$. These pooled distributions are element-weighted mixtures, not distributions of layer means.

Exact zeros are stored separately from the nonzero histogram. The native grid spans $[-8, 8]$ with 16,000 bins of nominal width 0.001; the figure combines ten bins at a time and divides by the actual floating-point bin width and the count of all captured elements. Consequently, a curve’s full integral is the recorded nonzero mass inside that grid, not one. Table 8 reports the zero mass and the fractions outside the displayed and native ranges. Underflow and overflow counts are retained rather than folded into edge bins. The displayed limits are $[-2.5, 3]$ for FFN and $[-4, 4]$ for attention in Figure 9. The largest omitted display tail is 6.6719%; the native-grid tail is at most 0.7607%. Figure 5 selects the two OL1 checkpoints at $\kappa = 0.5$, using the same density normalization and symlog scale, with FFN limits $[0, 3]$ and attention limits $[-4, 4]$.

The diagnostic reruns use FP32 parameters with FP16 autocast and no additional clipping. Their maximum absolute validation-loss difference from the retained reference is 6.89×10^{-5} , below the specified 5×10^{-4} tolerance; all 108 threshold-consistency checks pass. Small rounding differences

Table 5: All 54 trained endpoints. λ is pressure weight; κ is the trained threshold. $U_{\text{arch}} = \mathcal{S}_{\text{model}}/\mathcal{S}_{\text{model}}^{\text{max}}$ (A7) uses the same A7 reference within each model size, including for A0. Rounded zero sparsity need not be exactly zero.

Size	Recipe	Parameter	Loss	$\mathcal{S}_{\text{model}}$ (%)	U_{arch} (%)
14M	A0	—	5.2086	0.000	0.000
14M	A1-H	—	5.2696	2.714	9.061
14M	A1-H-L1	$\lambda = 0.05$	5.2061	3.142	10.489
14M	A1-H-L1	$\lambda = 0.1$	5.1655	3.336	11.139
14M	A1-H-L1	$\lambda = 0.5$	5.1127	3.788	12.645
14M	A1-H-L1	$\lambda = 1$	5.1023	3.949	13.185
14M	A1-H-OL1	$\lambda = 0.05$	5.1981	3.145	10.501
14M	A1-H-OL1	$\lambda = 0.1$	5.1594	3.339	11.146
14M	A1-H-OL1	$\lambda = 0.5$	5.1102	3.768	12.580
14M	A1-H-OL1	$\lambda = 1$	5.1212	3.938	13.149
14M	A4	$\kappa = 0$	5.4705	7.212	24.078
14M	A4	$\kappa = 0.01$	5.4665	7.414	24.752
14M	A4	$\kappa = 0.05$	5.4341	8.206	27.396
14M	A4	$\kappa = 0.1$	5.4197	8.953	29.891
14M	A4	$\kappa = 0.5$	5.6597	10.216	34.106
14M	A4-OL1	$\kappa = 0$	5.4583	7.810	26.075
14M	A4-OL1	$\kappa = 0.01$	5.4583	8.587	28.668
14M	A4-OL1	$\kappa = 0.05$	5.4898	10.456	34.910
14M	A4-OL1	$\kappa = 0.1$	5.5483	11.394	38.041
14M	A4-OL1	$\kappa = 0.5$	6.0380	12.713	42.446
14M	A7	$\kappa = 0$	5.4684	7.218	24.097
14M	A7	$\kappa = 0.01$	5.4588	7.618	25.433
14M	A7	$\kappa = 0.05$	5.4379	9.127	30.471
14M	A7	$\kappa = 0.1$	5.4287	10.425	34.805
14M	A7	$\kappa = 0.5$	5.7029	15.387	51.371
14M	A7-OL1	$\kappa = 0$	5.4802	7.054	23.551
14M	A7-OL1	$\kappa = 0.01$	5.4758	7.723	25.786
14M	A7-OL1	$\kappa = 0.05$	5.4628	9.863	32.929
14M	A7-OL1	$\kappa = 0.1$	5.4295	11.797	39.385
14M	A7-OL1	$\kappa = 0.5$	5.8294	27.483	91.755
70M	A0	—	4.0998	0.000	0.001
70M	A1-H	—	4.2227	10.066	20.366
70M	A4-OL1	$\kappa = 0$	4.8054	25.673	51.944
70M	A4-OL1	$\kappa = 0.01$	4.8229	26.224	53.060
70M	A4-OL1	$\kappa = 0.05$	4.8742	28.490	57.644
70M	A4-OL1	$\kappa = 0.1$	4.9439	30.574	61.861
70M	A4-OL1	$\kappa = 0.5$	5.3895	35.596	72.022
70M	A7-OL1	$\kappa = 0$	4.9412	23.562	47.674
70M	A7-OL1	$\kappa = 0.01$	4.9364	24.226	49.017
70M	A7-OL1	$\kappa = 0.05$	4.9637	26.524	53.666
70M	A7-OL1	$\kappa = 0.1$	4.9501	28.694	58.056
70M	A7-OL1	$\kappa = 0.5$	5.2159	40.602	82.150
410M	A0	—	4.5475	0.011	0.013
410M	A1-H	—	4.6513	21.050	24.127
410M	A4-OL1	$\kappa = 0$	5.6921	38.339	43.944
410M	A4-OL1	$\kappa = 0.01$	5.6914	39.149	44.873
410M	A4-OL1	$\kappa = 0.05$	5.6789	42.125	48.283
410M	A4-OL1	$\kappa = 0.1$	5.6789	45.688	52.367
410M	A4-OL1	$\kappa = 0.5$	5.1910	71.591	82.058
410M	A7-OL1	$\kappa = 0$	5.4263	41.121	47.133
410M	A7-OL1	$\kappa = 0.01$	5.4331	41.478	47.542
410M	A7-OL1	$\kappa = 0.05$	5.4679	44.053	50.494
410M	A7-OL1	$\kappa = 0.1$	5.4907	48.998	56.161
410M	A7-OL1	$\kappa = 0.5$	5.1207	80.616	92.401

Table 6: All 29 Pythia-14M contrasts, treatment minus reference. When both recipes have a swept parameter, its value is matched; A0 and A1-H are fixed controls.

Treatment – reference	Parameter	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
A1-H – A0	—	+0.0611	+2.714
A1-H-L1 – A1-H	$\lambda = 0.05$	-0.0635	+0.427
A1-H-L1 – A1-H	$\lambda = 0.1$	-0.1042	+0.622
A1-H-L1 – A1-H	$\lambda = 0.5$	-0.1569	+1.073
A1-H-L1 – A1-H	$\lambda = 1$	-0.1674	+1.235
A1-H-OL1 – A1-H-L1	$\lambda = 0.05$	-0.0081	+0.004
A1-H-OL1 – A1-H-L1	$\lambda = 0.1$	-0.0061	+0.002
A1-H-OL1 – A1-H-L1	$\lambda = 0.5$	-0.0025	-0.020
A1-H-OL1 – A1-H-L1	$\lambda = 1$	+0.0189	-0.011
A4 – A1-H	$\kappa = 0$	+0.2009	+4.498
A4 – A1-H	$\kappa = 0.01$	+0.1969	+4.700
A4 – A1-H	$\kappa = 0.05$	+0.1645	+5.492
A4 – A1-H	$\kappa = 0.1$	+0.1500	+6.239
A4 – A1-H	$\kappa = 0.5$	+0.3900	+7.501
A4-OL1 – A4	$\kappa = 0$	-0.0122	+0.598
A4-OL1 – A4	$\kappa = 0.01$	-0.0082	+1.173
A4-OL1 – A4	$\kappa = 0.05$	+0.0556	+2.251
A4-OL1 – A4	$\kappa = 0.1$	+0.1286	+2.441
A4-OL1 – A4	$\kappa = 0.5$	+0.3783	+2.498
A7 – A4	$\kappa = 0$	-0.0021	+0.006
A7 – A4	$\kappa = 0.01$	-0.0077	+0.204
A7 – A4	$\kappa = 0.05$	+0.0038	+0.921
A7 – A4	$\kappa = 0.1$	+0.0090	+1.472
A7 – A4	$\kappa = 0.5$	+0.0432	+5.171
A7-OL1 – A7	$\kappa = 0$	+0.0118	-0.163
A7-OL1 – A7	$\kappa = 0.01$	+0.0170	+0.106
A7-OL1 – A7	$\kappa = 0.05$	+0.0249	+0.736
A7-OL1 – A7	$\kappa = 0.1$	+0.0008	+1.372
A7-OL1 – A7	$\kappa = 0.5$	+0.1265	+12.096

Table 7: All matched A7-OL1 minus A4-OL1 comparisons. Negative loss changes and positive sparsity changes favor A7-OL1.

Size	κ	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.01	+0.0175	-0.863
14M	0.05	-0.0270	-0.593
14M	0.1	-0.1188	+0.402
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.01	+0.1135	-1.998
70M	0.05	+0.0895	-1.966
70M	0.1	+0.0062	-1.880
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.01	-0.2582	+2.329
410M	0.05	-0.2111	+1.929
410M	0.1	-0.1881	+3.310
410M	0.5	-0.0703	+9.024

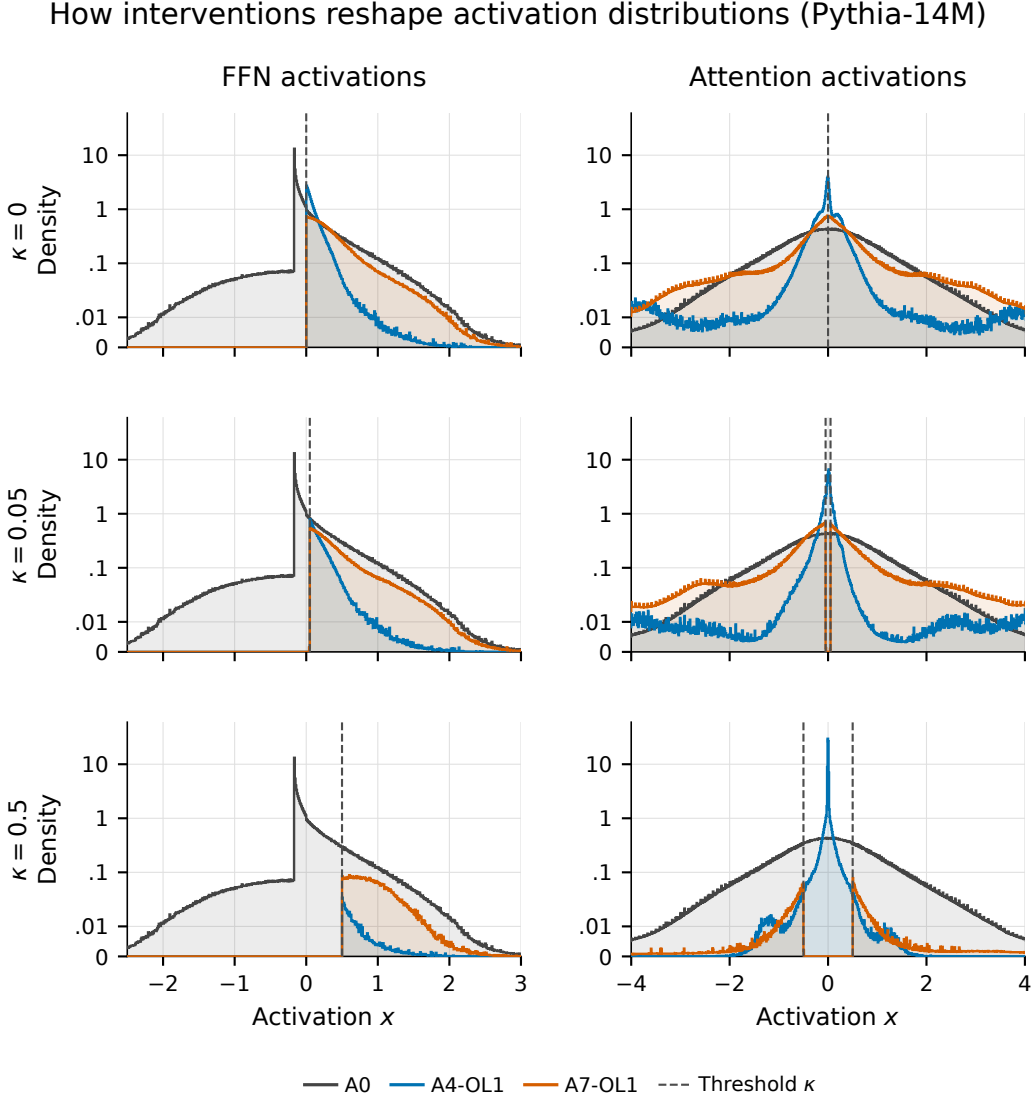


Figure 9: **Interventions reshape FFN and attention distributions differently. Exact zeros are omitted from the curves and reported in Table 8.** Columns pool FFN (h, m) and attention (q, k, v) activations; rows use trained thresholds $\kappa = 0, 0.05, 0.5$. A0 is repeated across rows. Counts are pooled across all layers and validation blocks, weighting $h : m$ by 80:20 and $q : k : v$ equally. Density divides nonzero bin counts by all captured elements and bin width, without renormalizing the visible range. The vertical scale is symlog, linear below 0.01; filled areas on this scale do not represent probability mass. Dashed lines mark $+\kappa$ for A4/A7 FFN thresholds and $\pm\kappa$ for A7 attention thresholds. Q/K are measured after RoPE. A4 leaves q, k, v without direct thresholding or pressure; A7 targets them directly. The seven checkpoints use no additional post-hoc clipping.

in exact-zero counts between evaluation passes do not change the displayed conclusions. Complete per-site and pooled histograms, counts and checkpoint hashes accompany the data release.

Table 8: Activation sparsity (the probability mass at zero) omitted from the density curves, and mass outside their displayed range (view tail) and the native $[-8, 8]$ histogram (grid tail). Grid tails are a subset of view tails, not an additional mass to add. All percentages divide integer counts by all captured elements. A0 is listed once although it is repeated in each figure row. These 14 groups come from seven checkpoints.

Recipe	κ	Group	Sparsity (%)	View tail (%)	Grid tail (%)
A0	—	FFN	< 0.01	0.1113	0.0000
A0	—	Attention	< 0.01	2.3929	0.4632
A4-OL1	0	FFN	67.22	0.0003	0.0000
A4-OL1	0	Attention	< 0.01	5.1717	0.6162
A4-OL1	0.05	FFN	86.71	0.0003	0.0000
A4-OL1	0.05	Attention	< 0.01	5.6601	0.7607
A4-OL1	0.5	FFN	99.31	0.0001	0.0000
A4-OL1	0.5	Attention	0.22	0.0003	0.0000
A7-OL1	0	FFN	64.52	0.0063	0.0000
A7-OL1	0	Attention	< 0.01	5.4657	0.4648
A7-OL1	0.05	FFN	75.29	0.0106	0.0000
A7-OL1	0.05	Attention	7.26	6.6719	0.6932
A7-OL1	0.5	FFN	93.64	0.0037	0.0000
A7-OL1	0.5	Attention	95.60	0.2396	0.0009

D.2 OPERATION CONTRIBUTIONS TO MODEL-WIDE SPARSITY

An operation’s contribution to S_{model} is its zero-product count divided by the full model product count. Summing these contributions recovers the model-wide fraction; summing local activation percentages would not. Table 9 gives this decomposition at $\kappa = 0.5$. At 14M, extending from A4-OL1 to A7-OL1 adds attention opportunity while reducing the contribution from some projections. At larger sizes, projection work carries more weight in the workload, so an equal change in activation sparsity need not yield an equal change in model-wide sparsity. Figure 10 displays the operation-level view at the same threshold and all three sizes.

Table 9: Contributions to $100S_{\text{model}}$ in percentage points at $\kappa = 0.5$. All six operations share the same model denominator within each size. The dense LM head contributes to that denominator but receives no zero credit. Differences in rounded entries can affect the displayed sum.

Operation	14M		70M		410M	
	A4-OL1	A7-OL1	A4-OL1	A7-OL1	A4-OL1	A7-OL1
QKV	3.173	2.430	8.820	6.231	17.563	16.992
QK	0.000	8.324	0.000	5.052	0.000	5.206
PV	0.057	8.449	0.011	5.371	0.008	5.858
Attention output	1.069	1.069	3.087	3.084	6.186	6.187
FFN up	4.139	2.939	11.329	8.526	22.938	21.503
FFN down	4.275	4.272	12.350	12.338	24.895	24.869
Total	12.713	27.483	35.596	40.602	71.591	80.616

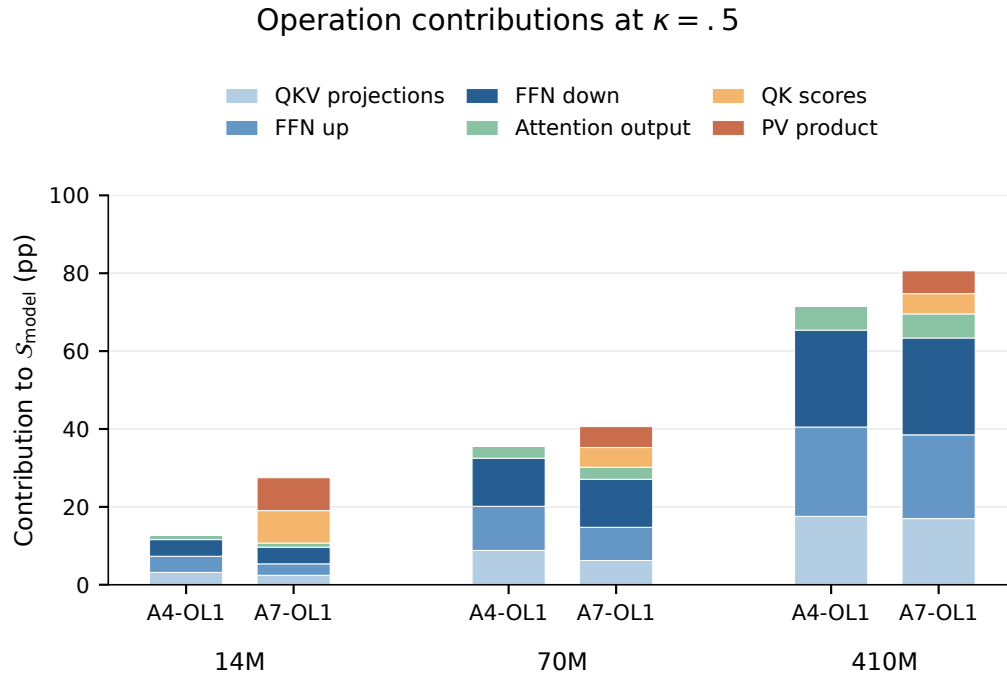


Figure 10: Operation accounting for A4-OL1 and A7-OL1 at $\kappa = 0.5$ across 14M, 70M and 410M. Zero-operand products are counted over causally valid positions and pooled across all 338 validation blocks. Each contribution uses the full model denominator, including the dense LM head without zero credit. At 14M, attention contributes more under A7-OL1, outweighing smaller contributions from several projections. The bars show logical multiplication opportunities, not runtime savings.

D.3 COMPLETE POST-HOC CLIPPING TRAJECTORIES

The complete release contains 540 evaluations: ten clipping targets for each of the 54 trained checkpoints. It joins 350 new evaluations with 190 retained evaluations matched by checkpoint content hash. Calibration uses the first ten complete training blocks in source order, separately for each layer’s a, m, h, z activations. Evaluation zeros values satisfying $|x| \leq t$ after the trained nonlinearities, without changing weights or A7’s internal attention nonlinearities. The target p controls a calibration quantile; it is not the achieved model-wide sparsity. In particular, an existing zero mass can make several targets identical.

Figures 11–13 retain the full loss range and every target, including dominated points and ties. Each path begins at its actually measured $p = 0$ evaluation. Small numerical differences from the canonical training endpoint can change strict nondominance without implying a substantive improvement. Numerical frontier memberships are stored separately from the lines connecting evaluated settings. All evaluations use complete validation with FP32 parameters, FP16 autocast, eager attention, batch one and $T = 2,048$.

14M training and post-hoc clipping

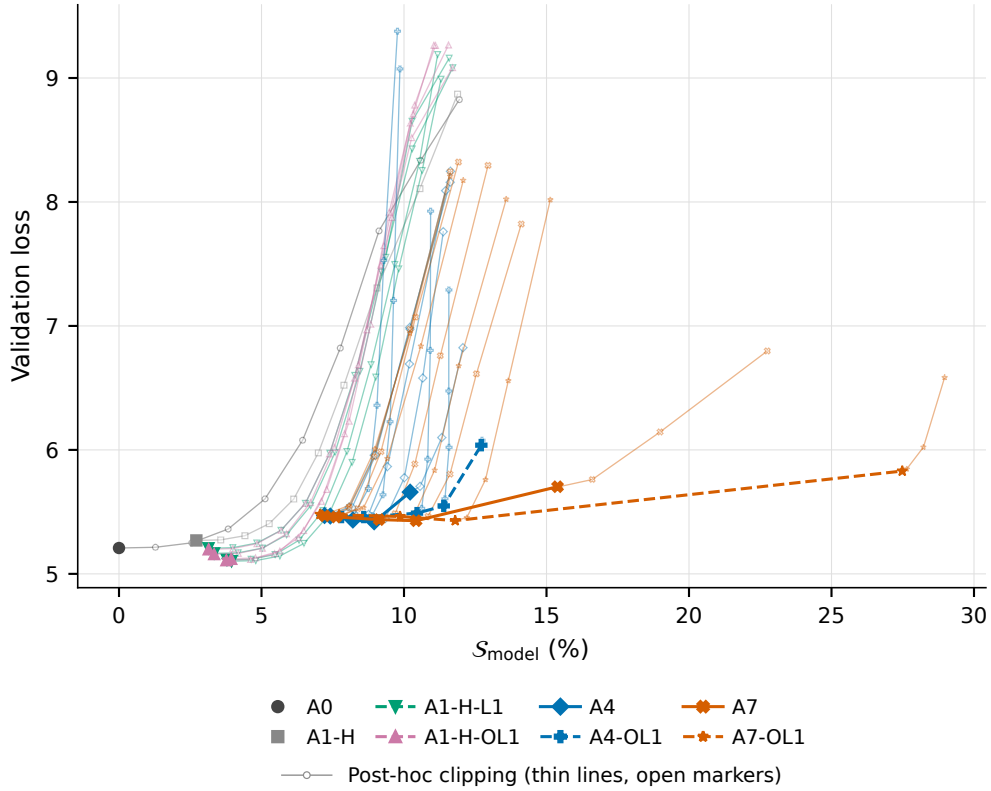


Figure 11: Complete 14M training and post-hoc clipping results: 30 trained endpoints and 300 clipping evaluations. Large filled markers denote trained endpoints; bold lines connect their settings, solid for unpressured recipes and dashed for L1/OL1. Each faint open-marker path follows one fixed checkpoint through $p = 0, 0.1, \dots, 0.9$. Colors and symbols identify its source recipe. The full loss range is shown. Connections order evaluated settings and do not interpolate attainable models or fitted Pareto envelopes.

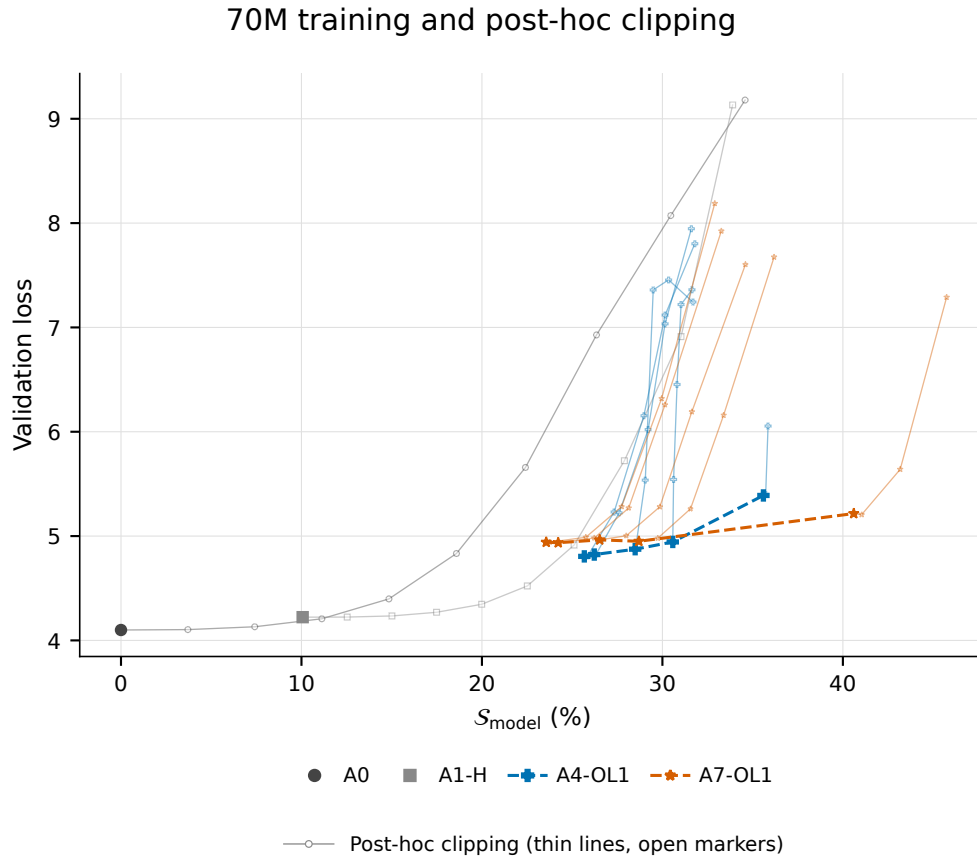


Figure 12: Complete 70M training and post-hoc clipping results: 12 trained endpoints and 120 clipping evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing clipping targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 11.

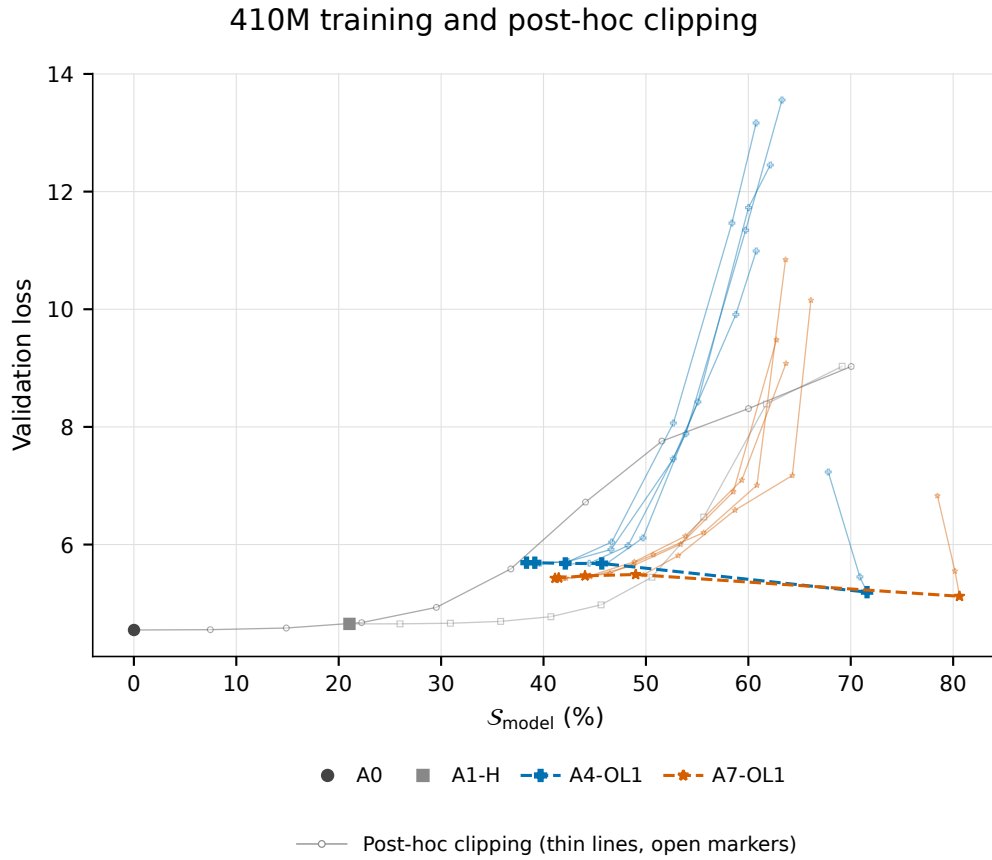


Figure 13: Complete 410M training and post-hoc clipping results: 12 trained endpoints and 120 clipping evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing clipping targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 11.

D.4 KERNEL QUALIFICATION AND ABLATIONS

The unchanged Sakana/TwELL control achieves $1.3001\times$ on its supported SparseLM-0.5B/H100 workload. Its raw projection primitive at the Pythia-14M width $N = 128$ instead gives relative L2 error 0.69055, exceeding the 0.02 bound; that test stops before full-model Pythia timing. This is a compatibility test, not an unchanged upstream speedup comparison on our model. The later P0 adapter changes signed-value packing, shape tails and numerical handling for Pythia. It retains dense attention and uses native projection fallback on A0. Table 10 reports this adapted baseline.

Historical proposals are replayed on four representative checkpoints; the plotted incumbent uses fixed A7-OL1, $\kappa = 0.5$. Final implementation and ablation summaries retain the 30-checkpoint manuscript cohort from the historical 35-checkpoint sweep. Five A4 checkpoints with pressure only at h are excluded because they do not implement the four-site pressure objective studied here. Search progress and the final sweep are separate measurements, explaining the small difference between their $1.7830\times$ and $1.7832\times$ maxima.

The coding-agent configuration records *gpt-6-astra* as the default model, not the model served for every request. This is one human-guided development trajectory and does not compare agent-assisted with human-only kernel development. Kernel measurements use Python 3.12.3, PyTorch 2.11.0+cu128, Transformers 5.12.1, CUDA 12.8 and nvcc 12.8.93. The supplementary protocol records dependency and frozen-source hashes.

Every candidate must produce finite outputs, with each logit satisfying $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, per-block relative output L2 error at most 0.02, and absolute pooled loss difference at most 0.001. Here $\hat{y}$ is the unmodified native eager reference. All three fresh processes must pass on all 338 validation blocks. Qualification is bounded numerical agreement on these inputs, not bitwise equivalence. Failed candidates are excluded from qualified speedup summaries. P0, an early projection adapter derived from the released kernels of Cetin et al. (2026), qualifies on only four checkpoints. Its average is not comparable to a 30-checkpoint mean or an unchanged upstream benchmark.

Timing uses the same checkpoint, inputs and CUDA-graph scaffold for native PyTorch/SDPA and each candidate. For each checkpoint, we take the geometric mean of 1,344 paired host-latency ratios (native divided by candidate): 64 inputs, seven randomized paired passes, and three fresh processes. Compilation, static weight preparation, graph capture and equal input staging are excluded. Preprocessing that depends on activations, embeddings, every block, final normalization and full vocabulary logits are included. There is no activation cache. Cohort summaries give each checkpoint equal weight.

K050 combines normalization and RoPE fusion, zero-fragment tensor-core bypass and sparse-row projection paths, using CUDA, CUTLASS, FlashAttention and PyTorch components. K049 is its searched predecessor; K050 adds fusion of the two input LayerNorms while preserving their separate affine parameters and thresholds. The all-skips-off control retains fusion while disabling both attention and projection skips; the attention-dense control retains projection skips. Their ratios are formed from separately measured, native-normalized timings. K050’s sparse paths improve 14 of 30 checkpoints relative to all-skips-off, with geometric mean ratio $1.0432\times$. Disabling attention skipping improves all 30: at the search checkpoint, instrumentation confirms that skipping bypasses 58.0% of eligible QK and 67.2% of PV tensor-core matrix multiply-accumulate (MMA) instructions, yet its overhead exceeds the saved time. Scalar zero-product fractions and skipped MMA fractions are different estimands.

The attention path tests whether either warp-distributed operand fragment contains only zeros, and issues an MMA only when both fragments contain a nonzero. Partial sparsity can therefore leave an entire instruction active. Zero detection, warp votes and branches recur inside the kernel; causal masking and softmax still execute. A zero QK score does not imply a zero softmax probability. The measurements do not identify a profitable joint-zero pattern or break-even threshold. They show that the skipped work does not repay these costs for the measured 14M shapes and hardware.

Table 10: Numerically qualified full-model speedups against native SDPA CUDA graphs. GM is the equal-checkpoint geometric mean; the range is across qualified checkpoints, not a confidence interval. All conditions use one RTX5090, BF16, batch one, 2,048 tokens and full vocabulary logits. P0’s four-checkpoint subset is shown only to document its limited qualification.

Implementation	Qualified	GM speedup	Range
K049	30/30	1.1141	0.9550–1.5284
K050	30/30	1.2340	1.0059–1.7832
K050, attention dense	30/30	1.2506	1.0185–1.8022
K050, all skips off	30/30	1.1829	1.0503–1.3628
P0 (qualified subset only)	4/30	0.8910	0.8180–1.0012

D.5 EVIDENCE AND DATA ACCOMPANYING THE DRAFT

The supplementary directory `supplementary-data/` contains all 54 training records with integer operation counts, all 540 clipping evaluations and frontier memberships, seven signed histograms with zero and tail counts, figure coordinates, and the 30-checkpoint runtime reduction. The included README defines fields and units. Copy hashes are recorded in `SOURCES.json`; `protocol.json` gives data preparation, training settings and implementation identities. Paths refer to the accompanying repository; the supplementary directory is not a standalone checkpoint or kernel release.